

設醺 Tsio Design Plan — 專案說明文件

本文件記錄這個網站從「純前端靜態網站」改造成「前端 + 後端 + 後台管理 + Docker 部署」的完整過程、技術細節與維護方式。最後更新：2026-07-29

目錄

1. [專案是什麼](#)
 2. [改造前的狀態](#)
 3. [改造的目標與技術決策](#)
 4. [整體架構](#)
 5. [改造歷程 \(分階段 \)](#)
 6. [資料結構與 API](#)
 7. [後台使用說明](#)
 8. [本機開發怎麼跑](#)
 9. [部署到 NAS \(Docker \)](#)
 10. [目前的上線狀態](#)
 11. [日常維護](#)
 12. [安全注意事項](#)
 13. [尚未完成 / 待辦](#)
 14. [檔案結構速查](#)
 15. [Git 版本紀錄](#)
-

1. 專案是什麼

「設醺 Tsio Design Plan」是一個設計活動的形象網站，內容包含展覽、工作坊、市集、講座、其他活動、以及[文章 \(報導 \)](#)等單元。

技術上是一個 **單頁應用 (SPA)**：使用者在瀏覽器裡切換頁面時不會整頁重新載入，由前端的路由 (react-router) 決定顯示哪個頁面。

2. 改造前的狀態

改造前，網站是**純前端**，所有文章資料寫在 `src/data/article.json` 裡，並且是用 `import` 的方式在**打包 (build)** 時就寫死進程式。

這造成一個根本限制：

想新增一篇文章，必須改原始碼 → 重新 build → 重新部署整包網站。沒有辦法「在網站上直接發文」。

此外還有一些既有的小問題（後面 code review 章節有處理）。

3. 改造的目標與技術決策

目標：讓網站能有一個**後台**，可以輸入文章標題、內文、上傳海報來發布文章，不用每次都改程式重新部署。

過程中做的幾個關鍵決策：

決策點	選擇	原因
後端語言	Node.js (不用 PHP)	與前端 React 同一種語言 (JavaScript) ，整個專案技術棧統一
後端與前端的關係	獨立的 Express API ，保留現有 Vite React	不用把現有網站重寫成 Next.js ，工程量最小
資料庫	SQLite (Node 24 內建的 <code>node:sqlite</code>)	單一檔案、免安裝資料庫服務、免編譯原生模組，Windows 與 Docker 都能跑
內文編輯方式	結構化區塊編輯器	忠實對應原本文章「小標 + 多段落」交錯的結構
部署方式	Docker 容器 ，跑在 Synology NAS	一個容器同時服務前端 + API + 圖片，好維護
對外路徑	子路徑 <code>/tsio-design/</code>	NAS 上 Bloomberg 已佔用網站根目錄，用路徑分流

4. 整體架構

瀏覽器（使用者）

|



單一 Node.js (Express) 伺服器

全部掛在 /tsio-design 前綴下：

GET /tsio-design/ → React 網站

GET /tsio-design/api/articles → 讀文章

POST /tsio-design/api/articles → 發文(需登入)

GET /tsio-design/uploads/xxx → 海報圖

SQLite (data.db)

uploads/ (海報圖檔)

- 前端：Vite 7 + React 19 + react-router 7 + Tailwind CSS v4
- 後端：Express 4 + Node 24 內建 SQLite
- 打包後：前端變成靜態檔（`dist/`），由 Express 一起吐出去
- 部署時整包裝進一個 **Docker 容器**

5. 改造歷程（分階段）

Phase 1 — 後端骨架

建立 `server/` 資料夾，用 Express 寫了一支後端：- 文章的列表 / 單篇 / 新增 / 刪除 / 更新 API - 單一管理員密碼登入，登入成功發一個 JWT token，之後發文都要帶這個 token - 使用 Node 24 內建 SQLite（免裝套件）- 寫了一支腳本 `import-articles.js`，把原本 `article.json` 的 8 篇文章匯入資料庫（保留舊網址 `ALUM-A001` 等）

Phase 2 — 前端主站改抓 API

把前端這幾個檔案從「`import` 打包死的 JSON」改成「即時 `fetch` 後端 API」：- 文章總覽頁、單篇文章頁、首頁「最新報導」區 - 順手修掉 `CardLayout` 原本靠「id 字串內容」判斷型別的脆弱寫法，改用明確欄位 - 加上「載入中 / 載入失敗 / 找不到文章」的狀態顯示

Phase 3 — `/admin` 後台

做了一個獨立的後台頁面（不含主站的 Nav / Footer）：- 密碼登入，token 存在瀏覽器 localStorage - 發文表單：標題、描述、日期、分類、上傳海報 - 結構化區塊編輯器：可以一塊一塊新增「小標」或「段落」、上下排序、刪除 - 已發布文章列表，可編輯與刪除

中途修正 — slug 乾淨化

新文章的網址原本會用中文標題產生一長串編碼過的亂碼網址，改成乾淨的短代碼 `a-{id}`（例：`/Articles/a-9`）。舊 8 篇維持原本的 `ALUM-A00x`。

Nav 顏色問題 + Code Review

- **Nav 標題消失問題**：原本 Nav 用 `mix-blend-difference`（差值混色），在中灰色背景（例如 Banner 照片，實測亮度 122）上，白色 logo 混色後也會變中灰、跟背景糊在一起看不見。→ 改成**自適應顏色**：偵測 Nav 下方是深色還是淺色區塊，自動切換白字或深色字（深色區塊 Banner、Footer 標記 `data-navcolor="white"`）。用 inline style 上色以避開 Tailwind 的樣式覆蓋問題。
- **Code review 修掉的 bug**：
- Footer 兩個死連結（「最新消息」原本指向不存在的 `/News`、「展覽」指向不存在的 `/Plan/Exhibition`）
- 補完三個原本空白的頁面：404 頁、報名頁、單一工作坊頁（接工作坊資料）
- 幾處 `class=` 寫錯成 HTML 而非 `className=`、一個不合法的 `<button><Link>` 巢狀、殘留的亂碼 class

統一佔位圖 + 文章編輯功能

- **統一佔位圖**：文章沒有上傳海報時，列表卡片與內頁大圖退回同一張預設圖，消除「列表一張、內頁另一張」的錯覺。
- **編輯功能**：後台每篇文章加「編輯」鈕，載入完整資料到表單，可修改標題 / 描述 / 日期 / 內文區塊 / 海報。更新時保留原網址（slug 不變）。

Phase 4 — 部署設定

- 新增 `Dockerfile` (多階段建置) 、 `docker-compose.yml` 、 `.dockerignore`
- 資料 (SQLite + 上傳圖片) 透過 `DATA_DIR` 環境變數指向掛載的 volume，重建容器不會遺失
- 匯入腳本改為「資料庫已有文章就略過」，避免正式環境的資料被覆蓋

子路徑部署

因為 NAS 上只有一個對外入口、Bloomberg 已佔用網站根目錄，把整個網站改成掛在 `/tsio-design/` 子路徑下：- 前端 vite `base` 與 router `basename` 改為 `/tsio-design` - 後端用 `BASE_PATH` 環境變數，把所有路由 (API / 圖片 / 靜態檔 / SPA fallback) 都掛在這個前綴下 - 新增 `src/lib/paths.js` 的 `url()`，讓所有 API 呼叫與海報網址自動接上這個前綴

6. 資料結構與 API

資料庫 (SQLite) 文章表 `articles`

欄位	說明
<code>id</code>	自動編號 (主鍵)
<code>slug</code>	網址用識別碼。舊文章沿用 <code>ALUM-A00x</code> ，新文章是 <code>a-{id}</code>
<code>title</code>	標題
<code>description</code>	摘要 / 描述
<code>date</code>	日期 (純文字，例： <code>04 08 2025</code>)
<code>location</code>	分類 / 標籤 (例： <code>校友特稿</code>)
<code>cover</code>	海報圖網址 (<code>/uploads/xxx</code>)，可為空
<code>blocks</code>	內文區塊，存成 JSON 字串
<code>created_at</code>	建立時間，用於排序 (新→舊)

blocks 的格式：一個陣列，每個元素是 `{ "type": "subtitle" 或 "paragraph", "content": "文字" }`。

API 端點 (都在 `/tsio-design` 前綴下)

方法	路徑	用途	需登入
POST	<code>/api/login</code>	管理員登入 · 回傳 token	X
GET	<code>/api/articles</code>	文章列表 (不含內文 · 省流量)	X
GET	<code>/api/articles/:slug</code>	單篇文章 (含內文區塊)	X
POST	<code>/api/articles</code>	新增文章 (可上傳海報)	✓
PUT	<code>/api/articles/:id</code>	更新文章 (保留 slug · 可換海報)	✓
DELETE	<code>/api/articles/:id</code>	刪除文章 (連海報檔一起刪)	✓

7. 後台使用說明

- 網址：`<你的網站網址>/tsio-design/admin`
- 登入：輸入管理員密碼 (設定在 `docker-compose.yml` 的 `ADMIN_PASSWORD`)
- 發新文章：填標題 → (可選) 描述、日期、分類 → (可選) 上傳海報 → 點「+ 小標 / + 段落」逐塊撰寫內文 → 送出
- 編輯文章：在下方文章清單點該篇的「編輯」，資料會帶入上方表單，改完按「更新文章」。網址不會變。
- 刪除文章：點「刪除」(會再確認一次)。
- 海報：編輯時若不選新檔，會保留原本的海報。

8. 本機開發怎麼跑

需要 Node.js (建議 24 以上)。

1. 安裝相依套件

```
npm install
```

2. (第一次) 把現有文章匯入本機資料庫

```
npm run import:articles
```

3. 開後端 (會讀 server/.env, 跑在 3001 埠)

```
npm run server:dev
```

4. 另開一個終端機, 開前端開發伺服器 (跑在 5181 埠)

```
npm run dev
```

開發網址: `http://localhost:5181/tsio-design/`

打包 (產生 dist/) :

```
npm run build
```

server/.env (本機設定, 不會進 git) 包含: `PORT`、`ADMIN_PASSWORD`、`JWT_SECRET`、`BASE_PATH`。

9. 部署到 NAS (Docker)

要放到 NAS 的檔案

複製整個專案, 但**不要**複製這些 (會自動重建或屬於本機資料): - `node_modules/` (Docker 會重裝) - `dist/` (Docker 會重建) - `.git/`、`server/.env`、`server/data.db`、`server/uploads/`

⚠ 專案資料夾**不要**放在 `/volume1/web/` (那是 Web Station 對外服務的目錄, 會讓 `docker-compose.yml` 裡的密碼被人下載)。目前放在 `/volume1/web_packages/docker/Tsio-Design-Plan`。

密碼設定 (`.env`)

密碼與密鑰放在專案資料夾的 `.env` 檔 (不進 git , `git pull` 不會覆蓋) :

```
cp .env.example .env    # 再編輯 .env 填入正式值
```

- `ADMIN_PASSWORD` : 後台登入密碼
- `JWT_SECRET` : 一長串隨機字元 (`openssl rand -hex 32`)

`docker-compose.yml` 其他設定已固定、不用動 : `BASE_PATH=/tsio-design` 、 `DATA_DIR=/data` + volume `./data:/data` 、埠對應 `8080:3001` 。

部署步驟 (Synology Container Manager)

1. 在專案資料夾裡先建一個 `data` 資料夾 (Docker 掛載需要它先存在) 。
2. Container Manager → 專案 → 新增 → 路徑選專案資料夾 → 來源選「使用現有的 `docker-compose.yml`」。
3. 建置 (第一次會下載 Node、裝套件、打包前端，需幾分鐘) 。
4. 狀態變「執行中」後，區網測試 `http://<NAS_IP>:8080/tsio-design/` 。

日後更新網站：把新程式碼覆蓋上去，Container Manager 對該專案重新「建置」即可。資料 (`data/`) 不受影響。

10. 目前的上線狀態

- 容器已在 NAS 上執行，NAS 的公開 IP 是 `120.126.18.18`，且 `8080` 埠對外是通的。
 - 目前可公開連線的網址： `http://120.126.18.18:8080/tsio-design/`
 - ⚠ 這是 HTTP (未加密) 。測試瀏覽沒問題，但後台登入的密碼會以明文傳輸，正式使用前應加上 HTTPS 。
 - 加 HTTPS 的方式 (尚未做) : 用 DSM 內建反向代理，在另一個 HTTPS 埠 (例如 `8443`) → `localhost:8080` ; 或等之後套上自己的網域，用反向代理搭配正式憑證。
-

11. 日常維護

- **新增 / 編輯 / 刪除文章**：直接用後台 `/tsio-design/admin`，不用碰程式。
 - **資料備份**：備份 NAS 上專案資料夾裡的 `data/`（裡面是 `data.db` 資料庫與 `uploads/` 海報圖）。這是唯一存放使用者資料的地方。
 - **更新程式**：在 NAS 專案資料夾 `git pull` 抓 GitHub 最新 → Container Manager 重新建置。`.env` 與 `data/` 不受影響。
 - **換部署路徑**：若要改子路徑或改成根目錄，需同時改兩處——`vite.config.js` 的 `base` 與 `docker-compose.yml` 的 `BASE_PATH`（兩者要一致）。
-

12. 安全注意事項

- **一定要設定密碼**：`ADMIN_PASSWORD` 與 `JWT_SECRET`（在 NAS 專案資料夾的 `.env`）。網站已對外公開。
 - **目前是 HTTP 明文**：後台登入前建議先加 HTTPS（見第 10 節），否則密碼有被攔截風險。
 - **專案資料夾別放在 `/volume1/web/`**：避免 `docker-compose.yml`（含密碼）被公開下載。
 - `server/.env`、`data.db`、上傳圖檔都已被 `.gitignore` / `.dockerignore` 排除，不會進版控或 image。
-

13. 尚未完成 / 待辦

- **HTTPS**：加內建反向代理或套自己的網域（目前是 HTTP）。
- **首頁分頁標題**：`index.html` 的 `<title>` 還是預設的「Vite + React」、`lang="en"`、`favicon` 是 Vite 圖示——正式上線前建議改成網站自己的。
- **重複的硬編碼內容**：市集、講座、其他活動、工作坊等頁面有大量複製貼上的重複區塊與 `Lorem ipsum` 假文，建議之後也資料化（像文章一樣可後台管理）。
- **聯絡頁社群連結**：`ContactPage` 的 Instagram / Threads 連結還是佔位字串，要填實際網址。
- **既有 8 篇文章沒有各自的海報**：匯入時沒有圖，目前都用預設佔位圖；可在後台逐篇編輯補上。

- 套件弱點：`npm install` 有回報一些相依套件弱點，尚未處理（避免動到現有前端版本）。
-

14. 檔案結構速查

Tsio-Design-Plan/

└ server/	★ 後端 (Node + Express + SQLite)
└ index.js	Express 入口 (掛在 BASE_PATH 前綴下)
└ config.js	設定 (埠、密碼、路徑、BASE_PATH、DATA_DIR)
└ db.js	SQLite 連線與建表
└ auth.js	密碼驗證與 JWT
└ routes/articles.js	文章 API (列表/單篇/新增/更新/刪除 + 海報上傳)
└ import-articles.js	首次把 8 篇文章匯入資料庫
└ .env	本機設定 (不進 git)
└ data.db、uploads/	本機的資料庫與上傳圖 (不進 git)
└ src/	★ 前端 (React)
└ main.jsx	進入點：路由設定、掛載 App
└ App.jsx / App.css	版面外框 (Nav + 內容 + Footer) 與全站樣式
└ admin/	後台 (登入、發文/編輯表單、文章管理、API 呼叫)
└ index_component/	首頁各區塊 (Nav、Banner、Footer、About、Article...)
└ routers/	各頁面 (文章、關於、報名、聯絡、404)
└ PlanFolder/	「計畫」底下的活動頁 (展覽、工作坊、市集、講座...)
└ small_component/	跨頁共用小元件 (卡片、標題、麵包屑、更多連結...)
└ hooks/	自訂 React hook (進場動畫 useScrollReveal)
└ lib/paths.js	把 API/圖片網址接上子路徑 base 的小工具
└ config/	設計參數 (motion 動畫、spacing 間距)
└ data/	活動用的本地 JSON (工作坊、展覽) + 文章匯入來源
└ assets/	圖片素材
└ banner/	首頁 Banner (desktop / mobile / tablet 三種尺寸)
└ icons/	UI 圖示 (logo.svg、社群、箭頭...)
└ imgs/	頁面內容示意圖 (文章封面、展覽、講座...)
└ photos/	首頁區塊用的照片 (About、圖庫、工作坊、計畫)
└ bg_gray.jpg	灰底背景紋理
└ logo_footer.png	Footer 用的 logo
└ public/vite.svg	靜態檔 (目前的 favicon)

└ Dockerfile	容器建置 (多階段)
└ docker-compose.yml	部署設定 (密碼、埠、volume、BASE_PATH)
└ .dockerignore / .gitignore	排除清單
└ vite.config.js	前端 base 與開發代理
└ eslint.config.js	程式碼檢查規則
└ package.json	相依套件與指令
└ 專案說明文件.md / 文章後台操作手冊.md	說明文件

15. Git 版本紀錄

專案已用 git 版控，每個階段都有一個 commit，隨時可還原：

Commit	內容
Initial commit	改造前的原始狀態 (還原點)
Phase 1	後端骨架 (Express + node:sqlite)
Phase 2	前端主站改抓後端 API
Phase 3	<code>/admin</code> 後台 (登入 + 結構化區塊發文 + 管理)
slug	新文章改用乾淨短代碼 <code>a-{id}</code>
Nav + bugfix	Nav 自適應顏色 + code review 修正
統一佔位圖 + 編輯	佔位圖統一 + 後台文章編輯功能
Phase 4	Docker 部署設定
子路徑	改為 <code>/tsio-design</code> 子路徑部署

查看完整紀錄：`git log --oneline`